

Definição das Ferramentas de Desenvolvimento

1. Introdução

O presente documento descreve as ferramentas, padrões e políticas utilizadas no desenvolvimento do app Bolinho, visando garantir organização, rastreabilidade, versionamento e controle das alterações realizadas durante o projeto.

2. Repositório do Projeto

O projeto utiliza a plataforma GitHub para controle de versão e gerenciamento do desenvolvimento.

Repositório

<https://github.com/Engenharia-de-Software2/bolinho.git>

3. Estrutura do Repositório

O repositório foi organizado da seguinte forma:

```
/frontend
/backend
  /data
  /include
  /src
/scripts
/tests
/docs
  /requisitos
  /casos-de-uso
  /diagramas
  /configuracao
.gitignore
README.txt
```

4. Ferramentas Utilizadas

Controle de Versão

- Git
- GitHub

Frontend

- HTML
- CSS
- JavaScript

Backend

- Node.js

Banco de Dados

- MySQL

Documentação

- Google Docs
- Draw.io

Gerenciamento

- GitHub Projects (Kanban)
 - Tabela de dependências
 - Gráfico de Gantt
 - Gráficos de barras para distribuição de tarefas
 - Milestones
 - GitHub Issues
-

5. Itens de Configuração

Os itens de configuração do projeto são:

- código-fonte frontend;
 - código-fonte backend;
 - diagramas UML;
 - documentos de requisitos;
 - casos de uso;
 - scripts SQL;
 - testes;
 - documentação técnica;
 - arquivos de configuração.
-

6. Política de Branches

O projeto utiliza branches por funcionalidade/tarefa.

Padrão de nomenclatura

feature/T1-documentacao-requisitos

feature/T10-cadastro-front

fix/login-error

docs/atualizacao-diagramas

Todas as branches são criadas a partir da branch principal main.

7. Política de Commits

Os commits seguem um padrão de identificação das tarefas do projeto.

Exemplo

[T1] Adiciona documentação de requisitos

[T10] Implementa cadastro frontend

[T15] Implementa autenticação login

8. Política de Merge

O merge das branches é realizado através de Pull Requests.

O fluxo adotado é:

Branch → Commit → Push → Pull Request → Merge

Os merges podem ser realizados pelos integrantes responsáveis pelo desenvolvimento do projeto.

9. Tratamento de Conflitos

Os conflitos de merge serão resolvidos pelos responsáveis das branches envolvidas.

Em caso de conflitos:

- será analisada a versão mais atual;
 - alterações críticas serão revisadas manualmente;
 - prioriza-se a integridade do sistema.
-

10. Versionamento

O projeto seguirá o padrão de versionamento semântico:

MAJOR.MINOR.PATCH

Exemplo

v1.0.0

v1.1.0

v2.0.0

11. Releases

Uma release será criada ao final de cada milestone importante do projeto.

As releases serão identificadas utilizando tags semânticas.

Exemplo

v1.0.0-beta

v1.0.0

12. Gerenciamento das Tarefas

O gerenciamento do projeto é realizado através do GitHub Projects utilizando a metodologia Kanban.

O quadro possui as seguintes etapas:

- Backlog;
 - A Fazer;
 - Em Desenvolvimento;
 - Revisão/Testes;
 - Concluído.
-

13. Dependência entre Artefatos

As dependências entre tarefas e documentos são controladas através de:

- GitHub Issues;
 - Milestones;
 - dependências “blocked by”;
 - Pull Requests vinculados às tarefas.
-

14. Conclusão

A utilização das ferramentas e políticas definidas busca garantir melhor organização, rastreabilidade, controle de mudanças e gerenciamento do desenvolvimento do aplicativo Bolinho, aplicando conceitos de Engenharia de Software e controle de configuração.